Feature Proposal

OscSchema -> Java/Kotlin 型生成

目的:

- `OscSchema` を唯一の仕様源にしながら、Java/Kotlinでプロパティ補完が効く開発体験を提供する
- 既存のOSCアプリから段階移行できるようにする

なぜ必要か

現状の `OscRuntime.send(...)` は `rawArgs: Map<String, Any?>` ベースなので、次の課題がある。

- キー typo がコンパイル時に検出されない
- 型ミスが実行時まで発見しづらい
- Java利用時の可読性が下がりやすい
- IDE補完が弱く、スキーマの意図がコード上で見えにくい

狙いは、既存API互換を維持しながら、型付きレイヤーを自動生成して上に重ねること。

提案する全体構成

- `osc-core`:
  - 既存の `OscSchema` / `OscRuntime` を維持
- `osc-codegen` (新規):
  - `OscSchema` から Kotlin/Javaコードを生成
- `osc-adapter-cli`:
  - `osc gen` を追加し手動生成をサポート
- `osc-gradle-plugin` (新規):
  - 外部Gradleプロジェクトで自動生成

ポイント:

- 仕様の真実源は常に `schema.kts` / `schema.yaml`
- 生成コードは破棄可能な成果物 (`build/generated/...`)

生成物の設計

1. `model`:

- メッセージごとの data class/POJO

2. `codec`:

- `toNamedArgs(): Map<String, Any?>`
- `fromNamedArgs(Map<String, Any?>)`

3. `runtime facade`:

- `OscRuntime` を型安全に呼ぶ薄いラッパー

この3層により、既存の `Map` ベースAPIと完全に相互運用できる。

具体例: schema

```yaml
messages:
  - path: /light/color
    name: light.color
    args:
      - { name: r, kind: scalar, type: int }
      - { name: g, kind: scalar, type: int }
      - { name: b, kind: scalar, type: int }

  - path: /mesh/points
    name: mesh.points
    args:
      - { name: pointCount, kind: scalar, type: int, role: length }
      - name: points
        kind: array
        lengthFrom: pointCount
        items:
          kind: tuple
          fields:
            - { name: x, type: int }
            - { name: y, type: int }
            - { name: z, type: float }
```

具体例: 生成されるKotlin型

```kotlin
data class LightColor(
    val r: Int,
    val g: Int,
    val b: Int,
) {
    fun toNamedArgs(): Map<String, Any?> = mapOf(
        "r" to r,
        "g" to g,
        "b" to b,
    )

    companion object {
        const val PATH: String = "/light/color"
        const val NAME: String = "light.color"

        fun fromNamedArgs(args: Map<String, Any?>): LightColor = LightColor(
            r = args["r"] as Int,
            g = args["g"] as Int,
            b = args["b"] as Int,
        )
    }
}
```

具体例: 送信側での利用

```
runtime.send(
    messageRef = LightColor.NAME,
    rawArgs = LightColor(255, 0, 128).toNamedArgs(),
    target = OscTarget("127.0.0.1", 9000),
)
```

具体例: LENGTH付き配列の生成方針

`role: length` + `lengthFrom` の組み合わせは、送信時に自動導出できる形を優先する。

```
data class MeshPoints(
    val points: List<Point3>,
) {
    val pointCount: Int get() = points.size

    data class Point3(
        val x: Int,
        val y: Int,
        val z: Float,
    )

    fun toNamedArgs(): Map<String, Any?> = mapOf(
        "pointCount" to pointCount,
        "points" to points.map { mapOf("x" to it.x, "y" to it.y, "z" to it.z) },
    )
}
```

これにより、呼び出し側が `pointCount` を二重管理しなくてよくなる。

外部Gradleプロジェクトからの使い方

```
plugins {
    kotlin("jvm") version "2.1.20"
    id("com.oscplatform.schema-codegen") version "0.4.0"
}

dependencies {
    implementation("com.oscplatform:osc-core:0.4.0")
    implementation("com.oscplatform:osc-transport-udp:0.4.0")
}

oscSchemaCodegen {
    schema.set(layout.projectDirectory.file("schema.yaml"))
    packageName.set("com.example.osc.generated")
    language.set("kotlin") // kotlin | java
}
```

外部Gradle連携で自動化されること

プラグインが以下を自動化する。

- `generateOscSources` タスク登録
- 出力先: `build/generated/sources/osc/main/<java|kotlin>`
- `sourceSets.main` へ生成ディレクトリ追加
- `compileKotlin` / `compileJava` が `generateOscSources` に依存

補完(POJO DX)は機能するか

結論: 機能する。

必要条件:

- 生成先を `sourceSets` に追加する
- コンパイル前に生成タスクを実行する
- 初回はIDEのGradle再同期を行う

この条件を満たせば、IDE上で通常の手書きクラスと同等に補完される。

既存OSCアプリからの移行手順

1. まず既存プロトコルを `schema.yaml` または `schema.kts` に定義
2. 主要メッセージだけ生成型へ置換
3. 未移行部分は `Map<String, Any?>` のまま共存
4. 受信処理も段階的に `fromNamedArgs(...)` へ移行

この方式なら、全置換を要求せずに安全に移行できる。

利点の整理

- 仕様変更が `OscSchema` 一箇所に集約される
- Java/Kotlinの型補完で開発速度が上がる
- typo/型ミスの多くをコンパイル時に検知できる
- CLI/MCP/Doc/Runtimeの整合性が保ちやすい
- 既存運用を止めずに段階導入できる

実装スコープ（初期版）

初期版で優先する実装:

- Kotlin生成を先行実装
- Java生成は次フェーズ
- `osc gen` と Gradleプラグインを同時提供
- サンプル1つを生成型で更新し、移行手順をREADME化

非目標（初期版ではやらない）:

- 既存 `OscRuntime` の破壊的変更
- 反射ベースの自動マッピング

実装プロンプト

以下をAI実装用のプロンプトとして利用する。

```
あなたはこのリポジトリ(osc-platform)のKotlin開発者です。以下の要件で実装してください。

目的:
OscSchema(schema.kts/schema.yaml)からKotlinの型安全クラスを生成し、外部Gradleプロジェクトから自動生成できるようにする。
既存のOscRuntime(Map<String, Any?> API)は互換維持する。

要件:
1. 新規モジュール osc-codegen を追加し、次を実装する。
  - OscSchema -> Kotlinコード生成器
  - 1メッセージ1 data class
  - toNamedArgs()/fromNamedArgs() 生成
  - PATH/NAME 定数生成
  - role=LENGTH + lengthFrom は送信時に自動導出し、二重管理を避ける
2. osc-adapter-cli に gen コマンドを追加する。
  - 例: osc gen --schema schema.yaml --package com.example.osc.generated --lang kotlin --out build/generated/sources/osc
  - 既存のCLIパース規約に合わせる
```

実装プロンプト（続き）

```
3. 新規モジュール osc-gradle-plugin を追加する。
  - plugin id: com.oscplatform.schema-codegen
  - 拡張設定: schema, packageName, language, outputDir
  - タスク generateOscSources を登録
  - sourceSets.main に生成先を追加
  - compileKotlin/compileJava が generateOscSources に依存
4. サンプルを1つ更新する。
  - sample/kotlin-quickstart-loopback を生成型利用に書き換える
  - READMEに移行前後の差分を記載
5. ドキュメント更新。
  - ルートREADMEに codegen の使い方を追加
  - 外部Gradleプロジェクト利用例を追加
```

実装プロンプト（最後）

```
制約:
- 既存テストを壊さない
- spotlessCheck/build が通ること
- 破壊的変更は禁止

作業後に実施:
- 変更ファイル一覧
- 実行したコマンドと結果
- 互換性への影響
- 今後の拡張ポイント
を簡潔に報告すること。
```